```
$ curl --cookie 'PHPSESSID=3fe1329aa9a3575373814ebf8aa59857' http://
localhost:8080/index.php
string(76) "a:2:{s:3:"cat";s:4:"Jess";s:6:"humans";a:2:{i:0;s:4:"Anna";i:1;
s:4:"Paul";}}"
```

... with the contents of the session file:

```
root@0d72be74121b:/usr/src/myapp# cat /tmp/sess_3fe1329aa9a3575373814e
bf8aa59857
data|a:2:{s:3:"cat";s:4:"Jess";s:6:"humans";a:2:{i:0;s:4:"Anna";i:1;s:4:
"Paul";}}
```

Session data need not be associative arrays; session serialization (and the `serialize()` function) will handle just about any data type, including objects (but not resources). Objects are serialized as their property names and values, including `protected` or `private` properties. The following example runs without the need for a web server, and in this case is better for it because some versions of `curl` might complain about "binary output" – all those metacharacters! It demonstrates object serialization using `serialize()` and `session_encode()`, which returns a serialized string representing the current state of `$_SESSION`:

```
class Foo
{
    public string $name = 'foo';
    private string $typeString = 'Instance of Foo';
    protected array $family = [
        'cat' => 'Jess',
        'humans' => ['Anna', 'Paul']
    ];
}

session_start();
print serialize(new Foo()) . PHP_EOL;

$_SESSION['data'] = new Foo();
print session_encode();
```